

程序设计训练 (Rust 语言)



第 1 讲：基本语法

韩文弢

清华大学

2025—2026 学年度夏季学期

本讲内容

1.1 Rust 语言介绍	2
1.2 第一次接触	6
1.3 基本语法	16
1.4 小结	53

1.1 Rust 语言介绍

1.1.1 Rust 语言

- 目标：构建**可靠**且**高效**的软件
- **高效** (performance)：没有运行时和垃圾收集器，代码的运行速度快，内存使用效率高，可用来开发对性能要求高的服务。
- **可靠** (reliability)：用类型系统和所有权模型来确保内存安全性和线程安全性，在编译时消除各种潜在的问题。
- **好用** (productivity)：有丰富的文档、友好的编译器（提供有用的错误信息）和一流的工具集，包括集成的包管理器和构建工具、支持各种编辑器的代码自动补全和类型查看功能、代码自动格式化工具等。
- 使用场景：命令行工具、网页应用、网络服务、嵌入式开发



图 1.1 Rust Logo



图 1.2 Ferris the Crab

1.1.2 Rust 语言发展历史

- 2006 年，Rust 作为 Graydon Hoare 的个人项目首次出现。
- 2009 年，Graydon Hoare 成为 Mozilla 雇员。
- 2010 年，Rust 首次作为 Mozilla 官方项目出现。同年，Rust 开始从初始编译（由 OCaml 写成）转变为自编译。
- 2011 年，Rust 成功完成移植，Rust 的自编译器采用 LLVM 作为其编译后端。
- 2012 年 1 月 20 日，第一个有版本号的预览版 Rust 编译器发布。
- 2013 年 4 月 4 日，Mozilla 基金会宣布将与三星集团合作开发浏览器排版引擎 Servo，此引擎将由 Rust 来实现。
- 2015 年 5 月 16 日，Rust 1.0.0 发布。
- 2021 年 2 月 8 日，AWS、华为、Google、微软以及 Mozilla 宣布成立 Rust 基金会。
- 2025 年 2 月 20 日，Rust 1.85.0 发布，Rust 2024 版本 (edition) 正式启用。

1.1.3 Rust 编写的软件

- Rust 语言工具链
- Redox 操作系统
- Linux 6.1 内核加入了用 Rust 语言编写部分代码的支持
- Servo 浏览器引擎
- Typst 排版系统
- lsd、bat、fd 等命令行工具
- rCore 教学操作系统
- MadFS 文件系统
-

1.2 第一次接触

1.2.1 Hello, Rust!

用 Rust 编写“Hello, world!”程序：

```
1 fn main() {  
2     println!("Hello, world!");  
3 }
```



【在线运行此代码】

编译：

```
1 $ rustc hello.rs
```

运行：

```
1 $ ./hello  
2 Hello, world!
```

知识点：

- 函数定义
- main 函数
- 输出（宏调用）
- 字符串
- 编译与运行

1.2.2 猜数游戏

通过一个简单的项目演示 Rust 的一些常见功能。项目要求：

- 程序随机产生一个 1—100 之间的秘密整数 n ，让用户来猜。
- 提示用户，输入一个想猜的数 x 。
- 如果 $x = n$ ，则猜数成功，打印祝贺信息并退出程序。
- 否则，根据 x 和 n 的大小关系输出提示信息，继续让用户来猜。

使用 Cargo 创建一个新的项目：

```
1 $ cargo new guessing_game
2 $ cd guessing_game
```

Cargo.toml¹ 的内容：

```
1 [package]
2 name = "guessing_game"
3 version = "0.1.0"
4 edition = "2024"
5
6 [dependencies]
```

[T] TOML

¹TOML (Tom's Obvious, Minimal Language) 是一种配置文件格式，常用于保存程序的配置信息。

src/main.rs 的内容与前面那个“Hello, world!”程序是一样的。编译并运行：

```
1 $ cargo run
2     Compiling guessing_game v0.1.0 (.../guessing_game)
3     Finished dev [unoptimized + debuginfo] target(s) in
  1.50s
4     Running `target/debug/guessing_game`
5 Hello, world!
```

- cargo run 命令可以自动进行编译并运行项目，便于快速迭代程序。
- 如果只要进行编译，可以用 cargo build 命令。

1.2.5 处理猜测

```
1  use std::io;
2
3  fn main() {
4      println!("Guess the number!");
5      println!("Please input your guess.");
6      let mut guess = String::new();
7      io::stdin()
8          .read_line(&mut guess)
9          .expect("Failed to read line");
10     println!("You guessed: {guess}");
11 }
```



Rust

知识点:

- 可变字符串类型
- 读取标准输入
- 错误处理
- 格式化输出

在 Cargo.toml 中加入 rand 包：

```
1 # ...
2 [dependencies]
3 rand = "0.10.2"
```

[T] TOML

编译：

```
1 $ cargo build
2     Updating crates.io index
3     Downloaded rand v0.10.2
4     ...
5     Compiling rand v0.10.2
6     Compiling guessing_game v0.1.0 (.../guessing_game)
7     Finished dev [unoptimized + debuginfo] target(s) in 2.53s
```

1.2.7 比较猜测情况

1.2 第一次接触

```
1  use rand::RngExt;
2  use std::cmp::Ordering;
3
4  let secret_number = rand::rng().random_range(1..=100);
5
6  let guess: u32 = guess.trim().parse()
7      .expect("Please type a number!");
8  println!("You guessed: {guess}");
9  match guess.cmp(&secret_number) {
10      Ordering::Less => println!("Too small!"),
11      Ordering::Greater => println!("Too big!"),
12      Ordering::Equal => println!("You win!"),
13  }
```



1.2.8 进行多次猜测

```
1  loop {
2      println!("Please input your guess.");
3      let mut guess = String::new();
4      io::stdin()
5          .read_line(&mut guess)
6          .expect("Failed to read line");
7      let guess: u32 = match guess.trim().parse() {
8          Ok(num) => num,
9          Err(_) => continue,
10     };
11     println!("You guessed: {guess}");
12     // ...
13 }
```



1.2.9 结束猜数过程

```
1  loop {
2      // ...
3      match guess.cmp(&secret_number) {
4          Ordering::Less => println!("Too small!"),
5          Ordering::Greater => println!("Too big!"),
6          Ordering::Equal => {
7              println!("You win!");
8              break;
9          }
10     }
11 }
```

 Rust

1.3 基本语法

1.3.1 变量绑定

- 使用 `let` 进行变量绑定 (variable binding) (注意不是赋值):

```
1 let x = 17;
```

 Rust

- 隐式类型 (types) 绑定: 由编译器根据上下文推断类型, 此处 `x` 为 `i32` 类型。
- 编译器不一定总能成功推断类型 (或者推断的结果不是用户想要的类型), 此时可以显式指定类型:

```
1 let x: i16 = 17;
```

 Rust

- 默认情况下变量是不可变的 (immutable); 如果要想变量可变, 需要使用 `mut` 修饰:

```
1 let x = 5;  
2 x += 1; // error: re-assignment of immutable variable x  
3 let mut y = 5;  
4 y += 1; // OK!
```

 Rust

1.3.2 变量绑定（续）

- 绑定可以被**掩盖** (shadowing), 也叫隐藏或重影:

```
1 let x = 17;  
2 let y = 53;  
3 // x is not mutable, but we're able to re-bind it  
4 let x = "Shadowed!";
```

 Rust

- 掩盖后的绑定 x 会一直存活到当前的作用域 (scope) 结束。
- 此时，第一个绑定已经丢失。
- 可以让同一含义不同类型的东西使用同一个变量名:

```
1 let mut cost = String::new();  
2 std::io::stdin().read_line(&mut cost).unwrap();  
3 let cost: f64 = cost.trim().parse().unwrap();
```

 Rust

```
1 const PI: f64 = 3.14159;  
2 const MAGIC: i32 = 42;
```



- 常量 (constants) 与不可变变量的区别
 - 编译时常数，在编译时可以确定。
 - 可以出现在任何作用域（包括全局作用域）。
 - 可以提前使用。

1.3.4 表达式

- 几乎所有东西都是表达式 (expressions): 会返回一个值 (value) 作为结果。
 - 例外: 变量绑定不是表达式。
- 单位元 (unit) 类型, 表示空, 写作 `()`。
 - 类型 `()` 只有一个可取的值: `()`。
 - `()` 是默认的回类型。
- 可以在表达式后加分号 `;` 来舍弃它的值, 这时它返回 `()`。
 - 因此, 如果一个函数以分号结尾, 它返回 `()`。

```
1 fn foo() -> i32 { 5 }  
2 fn bar() -> () { () }  
3 fn baz() -> () { 5; }  
4 fn qux()      { 5; }
```



- 由于基本上所有东西都是表达式，因此都可以绑定到变量：

```
1  let mut x = -5;
2  let y = if x > 0 {
3      "greater"
4  } else {
5      "less"
6  };
7  let z = loop {
8      x += 10;
9      if x > 5 {
10         break x;
11     }
12 };
```

 Rust

1.3.6 基本类型

- 布尔 `bool`: 两个值 `true/false`。
- 字符 `char`: 用单引号, 例如 `'R'`、`'计'`, 是 Unicode 的。
- 数值: 分为整数和浮点数, 有不同的大小和符号属性。
 - `i8`、`i16`、`i32`、`i64`、`i128`、`isize`
 - `u8`、`u16`、`u32`、`u64`、`u128`、`usize`
 - `f32`、`f64`
 - 其中 `isize` 和 `usize` 是指针大小的整数, 因此它们的大小与机器架构相关。
 - 字面值 (literals) 写为 `10i8`、`10u16`、`10.0f32`、`10usize` 等。
 - 字面值如果不指定类型, 且没有被自动推断为其他类型, 则默认整数为 `i32`, 浮点数为 `f64`。
- 数组 (arrays)、切片 (slices)、str 字符串 (strings)、元组 (tuples)
- 函数

1.3.7 数组

- 数组类型的形式为 `[T; N]`，例如 `[i32; 10]`。
 - `N` 是编译时常数 (compile-time constant)，也就是说数组的长度是固定的。
 - 运行时 (runtime) 访问数组元素会检查是否越界。
- 用 `[]` 来访问数组元素，数组下标从 0 开始。

```
1 let arr1 = [1, 2, 3]; // (array of 3 elements)
2 let arr2 = [2; 32];   // (array of 32 `2`s)
```



1.3.8 切片

- 切片类型的形式为 `&[T]`，例如 `&[i32]`。
- 切片表示引用数组中的一部分所形成的视图。
- 切片不能直接创建，需要从别的变量借用 (borrow)。
- 切片可以是可变的，也可以是不可变的。
- 如何知道一个切片是否有效？这是下一讲的内容。

```
1 let arr = [0, 1, 2, 3, 4, 5];  
2 let total_slice = &arr;           // Slice all of `arr`  
3 let total_slice = &arr[..];       // Same, but more explicit  
4 let partial_slice = &arr[2..5];   // [2, 3, 4]
```



1.3.9 字符串

- Rust 有两种字符串：String 和 &str。
- String 是在堆上分配空间、可以增长的字符序列。
- &str 是 String 的切片类型¹。
- 形如 "foo" 的字符串字面值都是 &str 类型的。

```
1 let s: &str = "galaxy";  
2 let s2: String = "galaxy".to_string();  
3 let s3: String = String::from("galaxy");  
4 let s4: &str = &s3;
```



¹&str 是没有大小的类型，编译时不知道大小，因此无法独立存在。

1.3.10 元组

- 元组是固定大小的、有序的、异构的列表类型。
- 可以通过下标来访问元组的分量，例如 `foo.0`。
- 可以使用 `let` 绑定来解构。

```
1 let foo: (i32, char, f64) = (72, 'H', 5.1);  
2 let (x, y, z) = (72, 'H', 5.1);  
3 let (a, b, c) = foo; // a = 72, b = 'H', c = 5.1
```



1.3.11 向量

`Vec<T>`

- 标准库提供的类型，可直接使用。
- `Vec` 是分配在堆上的、可增长的数组。
 - 类似 C++ 中的 `std::vector`、Java 中的 `java.util.ArrayList`。
- `<T>` 表示泛型，使用时代入实际的类型。
 - 例如，元素是 `i32` 类型的 `Vec` 写做 `Vec<i32>`。
- 使用 `Vec::new()` 或 `vec!` 宏来创建 `Vec`。
 - `Vec::new()` 是名字空间的例子，`new` 是定义在 `Vec` 结构体中的函数。

1.3.12 向量（续）

1.3 基本语法

```
1 // Explicit typing
2 let v0: Vec<i32> = Vec::new();
3
4 // v1 and v2 are equal
5 let mut v1 = Vec::new();
6 v1.push(1);
7 v1.push(2);
8 v1.push(3);
9
10 let v2 = vec![1, 2, 3];
11
12 // v3 and v4 are equal
13 let v3 = vec![0; 4];
14 let v4 = vec![0, 0, 0, 0];
```



1.3.13 向量（续）

```
1 let v2 = vec![1, 2, 3];  
2 let x = v2[2]; // 3
```



Rust

- 向量可以像数组一样使用 `[]` 来访问元素。
 - 在 Rust 中不能用 `i32/i64` 等类型的值作为下标访问元素。
 - 必须使用 `usize` 类型的值，因为 `usize` 保证和指针是一样长度的。
 - 其他类型要显式转换成 `usize`：

```
1 let i: i8 = 2;  
2 let y = v2[i as usize];
```



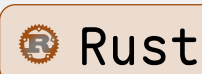
Rust

- 标准库中的向量有很多有用的方法，具体可以参见 [Rust 官方文档](#)。

1.3.14 类型转换

- 用 `as` 进行类型转换 (cast):

```
1 let x: i32 = 100;  
2 let y: u32 = x as u32;
```



- 一般来说，只能在可以安全转换的类型之间进行转换操作。
 - 例如，`[u8; 4]` 不能转换为 `char` 类型。
 - 有不安全的机制可以做这样的事情，代价是编译器就无法再确保安全性。

1.3.15 引用

- 在类型前面写 `&` 表示引用类型： `&i32`。
- 用 `&` 来取引用（和 C++ 类似）。
- 用 `*` 来解引用（和 C++ 类似）。
- 在 Rust 中，**引用保证是合法的**。
 - 合法性要通过编译时检查。
- 因此，Rust 中引用和一般意义的指针是不一样的。
- 引用的生命周期比较复杂，后面会详细讨论。

```
1 let x = 12;  
2 let ref_x = &x;  
3 println!("{}", *ref_x); // 12
```



1.3.16 条件语句

```
1  if x > 0 {  
2      10  
3  } else if x == 0 {  
4      0  
5  } else {  
6      println!("Not greater than zero!");  
7      -10  
8  }
```



Rust

- 与 C++ 不同，条件部分不需要用小括号括起来。
- 整个条件语句会当做一个表达式来求值，因此每个分支都必须是相同类型的表达式。
 - 如果想作为普通的条件语句来使用，可以令结果类型是 ()。

```
1  if x <= 0 {  
2      println!("Too small!");  
3  }
```



Rust

1.3.17 循环语句

- Rust 有三种循环：
 - while
 - loop
 - for
- break 和 continue 用于改变循环中的控制流。

1.3.18 while 循环语句

- while 的用法与 C++ 相同（和 if 一样，条件部分不需要用小括号括起来）：

```
1 let mut x = 0;
2 while x < 100 {
3     x += 1;
4     println!("x: {}", x);
5 }
```



1.3.19 loop 循环语句

- loop 相当于 while true, 或者是 C++ 中的 for (;;)。
- loop 循环中的 break 语句可以返回一个值, 作为整个循环的求值结果 (另外两种循环没有这个功能)。

```
1 let mut x = 0;
2 let y = loop {
3     x += 1;
4     if x * x >= 100 {
5         break x;
6     }
7 };
```



1.3.20 for 循环语句

- for 和 C++ 中的范围循环 for (auto x : v) 相似，使用迭代器 (iterators) 表达式：
 - `n..m` 创建一个从 `n` 到 `m` 半闭半开区间的迭代器。
 - `n..=m` 创建一个从 `n` 到 `m` 闭区间的迭代器。
 - 很多数据结构可以当做迭代器来使用，比如数组、切片，还有向量 `Vec` 等等。

```
1 // Loops from 0 to 9.
2 for x in 0..10 {
3     println!("{}", x);
4 }
5 let xs = [0, 1, 2, 3, 4];
6 // Loop through elements in a slice of `xs`.
7 for x in &xs {
8     println!("{}", x);
9 }
```



1.3.21 跳出多层循环

可以通过 `'label:` 和 `break 'label` 跳出多层循环。

```
1  let mut sum = 0;
2  'outer: for i in 1..10 {
3      for j in 1..10 {
4          sum += i * j;
5          if sum > 1000 {
6              break 'outer;
7          }
8      }
9  }
10 println!("sum = {sum}");
```



1.3.22 匹配语句

```
1 let x = 3;
2 match x {
3     1 => println!("one fish"), // <- comma required
4     2 => {
5         println!("two fish");
6         println!("two fish");
7     }, // <- comma optional when using braces
8     _ => println!("no fish for you"), // "otherwise" case
9 }
```



Rust

- 匹配语句由一个表达式 (x) 和一组 `value => expression` 的分支语句组成。
- 整个匹配语句被视为一个表达式来求值。
 - 与 `if` 类似，所有分支都必须是同样类型的值。
- 下划线 (`_`) 用于捕捉所有情况。

```
1 let x = 3;
2 let y = -3;
3 match (x, y) {
4     (1, 1) => println!("one"),
5     (2, j) => println!("two, {}", j),
6     (_, 3) => println!("three"),
7     (i, j) if i > 5 && j < 0 => println!("On guard!"),
8     (_, _) => println!(":<"),
9 }
```



- 匹配的表达式可以是任意表达式，包括元组和函数调用。
 - 构成模式 (patterns)。
 - 匹配可以绑定变量，_ 用来忽略不需要的部分。
- 为了通过编译，必须写穷尽的匹配模式。
- 可以用 if 来限制匹配的条件。

1.3.24 模式绑定

- 模式可以非常复杂，后面还会讲解到。
- 模式 (patterns) 也可以用来声明并绑定变量：

```
1 let (a, b) = ("foo", 12);
```



1.3.25 函数定义

```
1 fn foo(x: T, y: U, z: V) -> T {  
2     // ...  
3 }
```



- `foo` 是一个函数 (function), 有三个参数:
 - `T` 类型的参数 `x`
 - `U` 类型的参数 `y`
 - `V` 类型的参数 `z`
- 返回值的类型是 `T`。
- Rust 必须显式定义函数的参数和返回值的类型。
 - 实际上编译器是可以推断函数的参数和返回值的类型的, 但是 Rust 的设计者认为显式指定是一种更好的实践。

1.3.26 函数的返回

- 函数的最后一个表达式是它的返回值。
 - 可以用 `return` 来提前返回。

```
1  fn square(n: i32) -> i32 {  
2      n * n  
3  }  
4  fn squareish(n: i32) -> i32 {  
5      if n < 5 { return n; }  
6      n * n  
7  }  
8  fn square_bad(n: i32) -> i32 {  
9      n * n;  
10 }
```



- 最后一个无法通过编译，为什么？

1.3.27 宏

- 宏看起来像函数，但是名字以！结尾。
- 可以做很多有用的事情。
 - 宏的原理是在编译时产生代码。
- 宏调用的方式看起来和函数类似。
- 用户可以自己来定义宏。
- 很多常用工具是用宏来实现的。

1.3.28 print! 和 println!

- 用于输出文字信息。
- 使用 {} 来做字符串插入，{:?} 做调试输出。
 - 有些类型，例如数组和向量，只能用调试输出的方式来打印。
- {} 里可以加数字，表示第几个参数，新版本还可以把变量名写在 {} 里。

```
1 let x = "foo";  
2 print!("{}", {}, {}", x, 3, true);  
3 // => foo, 3, true  
4 println!("{:?}, {:?}", x, [1, 2, 3]);  
5 // => "foo", [1, 2, 3]  
6 let y = 1;  
7 println!("{0}, {y}, {0}", x);  
8 // => foo, 1, foo
```



- 使用与 `print! / println!` 相同的用法来创建 `String` 字符串。

```
1 let fmted = format!("{}", {:x}, {:?} ", 12, 155,  
  Some("Hello"));  
2 // fmted == "12, 9b, Some("Hello")"
```



1.3.30 dbg!

- 用于输出变量 / 表达式的值用于调试。
- 自身也是一个表达式，值就是内部表达式的值。

```
1 let x = 2;  
2 let y = dbg!(x * 2) + 3;  
3 dbg!(y);
```



```
1 [src/main.rs:3:13] x * 2 = 4  
2 [src/main.rs:4:5] y = 7
```

- 恐慌：显示提示信息，并退出当前任务。
- 是一种处理错误的方式，然而并不优雅。
- 如果有更好的处理方式，建议不要使用 `panic!`。

```
1  if x < 0 {  
2      panic!("Kaboom!");  
3  }
```



1.3.32 assert! 和 assert_eq!

- 如果条件 `condition` 不成立, `assert!(condition)` 会导致恐慌。
- 如果 `left != right`, `assert_eq!(left, right)` 会导致恐慌。
- 非常有用, 用于测试和捕捉非法条件。

```
1  #[test]
2  fn test_something() {
3      let actual = 1 + 2;
4      assert!(actual == 3);
5      assert_eq!(3, actual);
6  }
```



- 用于表示不会达到的代码分支。
- 如果运行到就会导致恐慌。
- 可用来追踪意料之外的问题。

```
1  if false {  
2      unreachable!();  
3  }
```



- `panic!("not implemented")` / `panic!("not yet implemented")` 的简写
- 可用于标注还没有实现的功能（例如作业里要补全的地方）
- 在语义上，`todo!` 传达了一种“以后会实现”的意思，而 `unimplemented!` 则没有

```
1 fn sum(x: Vec<i32>) -> i32 {  
2     todo!();  
3 }
```



1.3.35 注释

- 注释 (comments) 分为块注释 `/* ... */` 和行注释 `// ...`。
- 三个 `/` 开头的行注释是文档字符串注释 (docstring comments)。

```
1  /// Triple-slash comments are docstring comments.
2  ///
3  /// `rustdoc` uses docstring comments to generate
4  /// documentation, and supports Markdown formatting.
5  fn foo() {
6      // Double-slash comments are normal.
7
8      /* Block comments
9       * also exist /* and can be nested! */
10     */
11 }
```



1.3.36 代码风格

- 命名规范
 - 大驼峰式命名法 (UpperCamelCase): 类型名、特型名
 - 蛇形命名法 (snake_case): 变量名、函数名、模块名、宏名
 - 大写蛇形命名法 (SCREAMING_SNAKE_CASE): 常量名、静态变量名
- 缩进与空白
 - Rust 的缩进用 4 个空格字符。
 - 符号前后的空格
 - 操作符前后各有一个空格, 例如 `x + 1`。
 - 分隔符后面有一个空格, 例如 `f(x, 1)`。
- 注释之道
 - 写有意义的注释
 - 写文档注释 (`///`、`//!`)
 - 尽量用英文写注释

1.4 小结

1.4.1 本讲小结

- Rust 语言介绍
- 简单程序的编译和运行
- Cargo 的初步使用
- Rust 的基本语法

下一讲：所有权与结构化数据